

Senior Project Final Report (mlreport)

API Imports

`Report` creates a report for a single model. `ComparisonReport` combines two or more built reports into one model-comparison report.

```
# If running in Google Colab, uncomment this install cell.  
# !rm -rf /content/mlreport  
# !git clone https://github.com/Lucas-Summers/mlreport.git  
# !pip install /content/mlreport
```

```
from mlreport import ComparisonReport, Report
```

API Flow

Most reports follow the same sequence:

1. Create a `Report` around a fitted model.
2. Add evaluation data with `add_split()` or `add_crossval()`.
3. Optionally attach hyperparameter search results with `add_search()`.
4. Call `build()` to compute metrics and plots.
5. Export the report to text, HTML, Markdown, JSON, or PDF.

1. Create a Report

For scikit-learn models, `mlreport` usually detects whether the model is for classification or regression. For custom models, pass `model_type` and `model_params` explicitly.

```
report = Report(  
    model, # fitted sklearn-compatible or custom model  
    title="Model Report",  
    author="Your Name",  
    description="Optional description of this model or experiment.",  
    theme="light", # or "dark"  
    cmap="viridis", # any matplotlib colormap  
    model_type=None, # optional: "classification" or "regression"
```

```
model_params=None, # optional dict of parameters to display
)
```

2. Add Evaluation Data

Choose one evaluation style. A single report should use either named train/test-style splits or cross-validation, not both.

Option A: Train/Test Splits

```
report.add_split("train", X_train, y_train)
report.add_split("test", X_test, y_test)

# If predictions are already computed:
report.add_split("test", X_test, y_test, y_pred_test)
```

Option B: Cross-Validation

```
report.add_crossval(X, y, cv=5)
report.add_crossval(X, y, cv=cv)
report.add_crossval(X, y, y_pred=y_pred_cv, cv=cv)
report.add_crossval(X, y, y_pred=y_pred_cv, fold_ids=fold_ids)
```

3. Add Hyperparameter Search Results

If you used `GridSearchCV` or `RandomizedSearchCV`, attach the fitted search object before building the report.

```
report.add_search(search_cv)
```

The current tuning plots support exactly two tuned hyperparameters.

4. Build Metrics and Plots

```
report.available_metrics()
report.available_plots()

report.build(
    exclude_metrics=[],
    exclude_plots=[],
)
```

5. Export the Report

```
report.to_txt()
report.to_html("report.html")
report.to_md("report.md")
report.to_json("report.json")
report.to_pdf("report.pdf")
```

6. Compare Models

Build individual reports first, then pass them to `ComparisonReport`.

```
baseline_report = (
    Report(model_a, title="Baseline")
    .add_split("test", X_test, y_test)
    .build()
)

candidate_report = (
    Report(model_b, title="Candidate")
    .add_split("test", X_test, y_test)
    .build()
)

comparison = ComparisonReport(
    reports=[baseline_report, candidate_report],
    title="Model Comparison",
    split="test",
    theme="light",
).build()

comparison.to_pdf("comparison.pdf")
```

API Reference

`mlreport.Report`

```
class mlreport.Report(
    model,
    title: str | None = None,
    author: str | None = None,
    description: str | None = None,
    theme: str = "light",
    cmap: str = "viridis",
    model_type: str | None = None,
    model_params: dict | None = None,
)
```

Build and render an evaluation report for one fitted model or for precomputed predictions from one model.

`Report` supports two mutually exclusive evaluation-data flows:

- Add named data splits with `add_split`.
- Add one cross-validation prediction set with `add_crossval`.

After adding evaluation data, call `build` before exporting or calling `to_dict`.

Parameters

`model` : object

- Fitted scikit-learn-compatible estimator or custom model object. If

predictions are not passed explicitly to `add_split` or `add_crossval`, the model must expose a callable `predict(X)` method.

`title` : str | None, default=None

- Optional report title shown in rendered output.

`author` : str | None, default=None

- Optional report author shown in rendered output.

`description` : str | None, default=None

- Optional prose description shown in rendered output and in comparison cards.

`theme` : str, default="light"

- Render theme name. Built-in themes include "light" and "dark".

`cmap` : str, default="viridis"

- Matplotlib colormap name used for generated plots.

`model_type` : str | None, default=None

- Explicit model type for custom models. Accepted explicit values are

"classification", "classifier", "regression", and "regressor". If

omitted, scikit-learn type checks are used.

`model_params` : dict | None, default=None

- Optional parameter mapping to display in the report. If omitted,

`model.get_params()` is used when available. Custom models without `get_params()` should pass this argument.

Attributes

`model` : object

- Model object passed to the constructor.

`model_type` : str | None

- Explicit model type label, if supplied.

`model_params` : dict | None

- Explicit model parameter mapping, if supplied.

`title` : str | None

- Report title metadata.

`author` : str | None

- Report author metadata.

`description` : str | None

- Report description metadata.

`theme` : str

- Theme used by text, HTML, Markdown, and PDF renderers.

`cmap` : str

- Colormap used for plots.

Methods

`Report.available_metrics`

```
Report.available_metrics() -> None
```

Print the metric IDs and display names available for the report's detected model type. The method prints to standard output and does not return a value.

Use these IDs in `build(exclude_metrics=[...])`.

Classification metric IDs:

- `accuracy` : Accuracy
- `precision_macro` : Precision (Macro)
- `recall_macro` : Recall (Macro)
- `f1_macro` : F1 Score (Macro)
- `precision_weighted` : Precision (Weighted)
- `recall_weighted` : Recall (Weighted)
- `f1_weighted` : F1 Score (Weighted)

Regression metric IDs:

- `r2` : R2 Score
- `mse` : Mean Squared Error
- `rmse` : Root Mean Squared Error
- `mae` : Mean Absolute Error
- `max_error` : Max Error
- `median_ae` : Median Absolute Error

Report.available_plots

```
Report.available_plots() -> None
```

Print the plot IDs and display names available for the report's detected model

type. The method prints to standard output and does not return a value.

Use these IDs in `build(exclude_plots=[...])`.

Classification plot IDs:

- `confusion_matrix` : Confusion Matrix
- `class_distribution` : Class Distribution
- `predicted_class_distribution` : Predicted Class Distribution
- `per_class_metrics` : Per-Class Metrics

Regression plot IDs:

- `predicted_vs_actual` : Predicted vs Actual
- `residuals` : Residuals vs Predicted
- `residual_hist` : Residual Distribution
- `qq` : Q-Q Plot

Report.add_split

```
Report.add_split(  
    name: str,  
    X: np.ndarray,  
    y: np.ndarray,
```

```
y_pred: np.ndarray | None = None,  
) -> Report
```

Add one named data split to a standard train/test style report.

Parameters

`name : str`

- Split identifier, for example `"train"`, `"val"`, or `"test"`.

`X : np.ndarray`

- Feature matrix for the split.

`y : np.ndarray`

- Ground-truth target values for the split.

`y_pred : np.ndarray | None, default=None`

- Optional predictions for the split. When omitted, `model.predict(X)` is called.

Returns

`Report`

- The same report instance, enabling method chaining.

Notes

Call `add_split` once per split that should appear in the report. The split data is converted to NumPy arrays internally. A report cannot mix `add_split` and `add_crossval`.

`Report.add_crossval`

```
Report.add_crossval(  
    X: np.ndarray,  
    y: np.ndarray,  
    y_pred: np.ndarray | None = None,  
    cv: Any | None = None,  
    fold_ids: np.ndarray | None = None,  
) -> Report
```

Add a cross-validation prediction set to the report.

Parameters

`X : np.ndarray`

- Full feature matrix used for cross-validation.

`y : np.ndarray`

- Full target vector.

`y_pred : np.ndarray | None, default=None`

- Optional out-of-fold predictions aligned one-to-one with `y`. When omitted, predictions are generated with scikit-learn `cross_val_predict`.

`cv : Any | None, default=None`

- Optional cross-validation splitter, fold count, or iterable of `(train_idx, test_idx)` pairs. Required when `y_pred` is omitted.

`fold_ids : np.ndarray | None, default=None`

- Optional fold identifier for each row. Cannot be used together with `cv`. Useful when `y_pred` was computed elsewhere but fold-level summaries are still desired.

Returns

Report

- The same report instance, enabling method chaining.

Notes

Cross-validation reports use one internal split named `"cv"`. When fold IDs are available, metric values for the `cv` split include fold scores plus summary statistics: `mean`, `std`, `min`, and `max`. A report cannot mix `add_crossval` and `add_split`.

Report.add_search

```
Report.add_search(search_cv) -> Report
```

Add fitted scikit-learn hyperparameter search results for tuning summaries and tuning plots.

Parameters

`search_cv` : object

- Fitted search object such as `GridSearchCV` or `RandomizedSearchCV`. The object must expose a non-empty `cv_results_` mapping.

Returns

Report

- The same report instance, enabling method chaining.

Notes

`mlreport` currently supports tuning visualizations for exactly two searched hyperparameters. It selects the score column in this order:

1. `mean_test_<refit_metric>` when `search_cv.refit` is a string and the column exists.
2. `mean_test_score` when available.
3. The first sorted `mean_test_*` column.

Report.build

```
Report.build(  
    exclude_metrics: list[str] | None = None,  
    exclude_plots: list[str] | None = None,  
) -> Report
```

Compute metrics and generate plots for the data added with `add_split` or `add_crossval`.

Parameters

`exclude_metrics` : list[str] | None, default=None

- Metric IDs to skip. Use `available_metrics()` to discover valid IDs.

`exclude_plots` : list[str] | None, default=None

- Plot IDs to skip. Use `available_plots()` to discover valid IDs.

Returns

Report

- The same report instance, enabling method chaining.

Notes

`build()` must be called before any renderer method or `to_dict()`.

Report.to_txt

```
Report.to_txt(path: str | None = None) -> Report
```

Render the built report to plain text.

Parameters

`path : str | None`, default=`None`

- Optional output file path. When omitted, the text report is printed to

standard output.

Returns

Report

- The same report instance, enabling method chaining.

Notes

Plain-text output excludes plots and tuning plot figures.

Report.to_html

```
Report.to_html(path: str) -> Report
```

Render the built report to HTML.

Parameters

`path : str`

- Output file path for the HTML report.

Returns

Report

- The same report instance, enabling method chaining.

Notes

Plot images are embedded in the HTML as base64 image data.

Report.to_pdf

```
Report.to_pdf(path: str) -> Report
```

Render the built report to PDF.

Parameters

`path : str`

- Output file path for the PDF report.

Returns

`Report`

- The same report instance, enabling method chaining.

Notes

PDF rendering uses the same report context as HTML, with plots embedded as image data before PDF generation.

Report.to_json

```
Report.to_json(path: str) -> Report
```

Render the built report to JSON.

Parameters

`path : str`

- Output file path for the JSON report.

Returns

`Report`

- The same report instance, enabling method chaining.

Notes

JSON output excludes plot figures and tuning plot figures because Matplotlib figures are not JSON-serializable.

Report.to_md

```
Report.to_md(path: str, image_dir: str | None = None) -> Report
```

Render the built report to Markdown.

Parameters

`path : str`

- Output file path for the Markdown report.

`image_dir : str | None, default=None`

- Directory where exported plot images are written. Defaults to an `images` directory next to `path`.

Returns

`Report`

- The same report instance, enabling method chaining.

Notes

Markdown output writes plots as PNG files and links to those files from the generated Markdown. Tuning plots are written with a `tuning_` filename prefix.

Report.to_dict

```
Report.to_dict() -> dict
```

Return the built report as a renderer-friendly dictionary.

Returns

`dict`

- Report payload containing `meta`, `model`, `data`, `metrics`, `plots`, and `tuning` sections.
- Example:

```
{
  "meta": {
    "title": ...,
    "author": ...,
    "description": ...,
    "generated_at": "YYYY-MM-DD HH:MM",
  },
  "model": {
    "name": ...,
    "type": "Classification" | "Regression" | ...,
    "sklearn": "True (v...)" | "False",
    "params": {...},
  },
  "data": {
    "features": ...,
    "splits": {...},
    "total": ...,
    "cv_folds": ...,
    "is_crossval": ...,
    "class_distribution": {...},
    "class_percentages": {...},
  },
  "metrics": {...},
  "plots": {...},
  "tuning": {
    "summary": ...,
    "plots": {...},
  },
}
```

mlreport.ComparisonReport

```
class mlreport.ComparisonReport(
    reports: list[Report],
    title: str | None = None,
    author: str | None = None,
    description: str | None = None,
    split: str | None = None,
    theme: str = "light",
    cmap: str = "viridis",
)
```

Build and render a comparison report from two or more already-built

[Report](#) instances.

The first report in `reports` is treated as the baseline when deltas are shown.

All compared reports must have the same model type and at least one compatible scalar metric on the selected split.

Parameters

`reports : list[Report]`

- Built `Report` instances to compare. Must contain at least two reports.

`title : str | None, default=None`

- Optional comparison report title.

`author : str | None, default=None`

- Optional report author.

`description : str | None, default=None`

- Optional comparison description.

`split : str | None, default=None`

- Optional split name to compare across all reports, for example `"test"` or `"cv"`. If omitted, `"test"` is preferred when available, otherwise `"cv"` is used.

`theme : str, default="light"`

- Render theme name. Built-in themes include `"light"` and `"dark"`.

`cmap : str, default="viridis"`

- Matplotlib colormap name used for comparison plots.

Attributes

`reports : list[Report]`

- Source reports being compared.

`title : str | None`

- Comparison title metadata.

`author : str | None`

- Comparison author metadata.

`description : str | None`

- Comparison description metadata.

`split : str | None`

- Requested comparison split, if supplied.

`theme : str`

- Theme used by text, HTML, Markdown, and PDF renderers.

`cmap : str`

- Colormap used for generated comparison plots.

Methods

ComparisonReport.build

```
ComparisonReport.build() -> ComparisonReport
```

Validate the source reports and build comparison rows and plots.

Returns

`ComparisonReport`

- The same comparison instance, enabling method chaining.

Notes

`build()` calls `to_dict()` on each source report, so every source report must already be built. If duplicate model class names appear, the comparison assigns unique display keys such as `"RandomForestClassifier (1)"` and `"RandomForestClassifier (2)"`.

For classification comparisons, generated comparison plot groups currently include:

- `confusion_matrix`
- `per_class_metrics`

For regression comparisons, generated comparison plot groups currently include:

- `predicted_vs_actual`
- `residual_hist`

ComparisonReport.to_txt

```
ComparisonReport.to_txt(path: str | None = None) -> ComparisonReport
```

Render the built comparison report to plain text.

Parameters

`path : str | None, default=None`

- Optional output file path. When omitted, the text report is printed to standard output.

Returns

`ComparisonReport`

- The same comparison instance, enabling method chaining.

`ComparisonReport.to_html`

```
ComparisonReport.to_html(path: str) -> ComparisonReport
```

Render the built comparison report to HTML.

Parameters

`path : str`

- Output file path for the HTML comparison report.

Returns

`ComparisonReport`

- The same comparison instance, enabling method chaining.

Notes

Comparison plots are embedded in the HTML as base64 image data.

`ComparisonReport.to_pdf`

```
ComparisonReport.to_pdf(path: str) -> ComparisonReport
```

Render the built comparison report to PDF.

Parameters

`path : str`

- Output file path for the PDF comparison report.

Returns

ComparisonReport

- The same comparison instance, enabling method chaining.

ComparisonReport.to_json

```
ComparisonReport.to_json(path: str) -> ComparisonReport
```

Render the built comparison report to JSON.

Parameters

path : str

- Output file path for the JSON comparison report.

Returns

ComparisonReport

- The same comparison instance, enabling method chaining.

ComparisonReport.to_md

```
ComparisonReport.to_md(path: str, image_dir: str | None = None) -> ComparisonReport
```

Render the built comparison report to Markdown.

Parameters

path : str

- Output file path for the Markdown comparison report.

image_dir : str | None, default=None

- Directory where exported comparison plot images are written. Defaults to an `images` directory next to `path`.

Returns

ComparisonReport

- The same comparison instance, enabling method chaining.

Notes

Markdown output writes comparison plot images as PNG files and links to those files from the generated Markdown.

ComparisonReport.to_dict

```
ComparisonReport.to_dict() -> dict
```

Return the built comparison report as a renderer-friendly dictionary.

Returns

dict

- Comparison payload containing meta, comparison, models, and metrics

sections.

- Example:

```
{
  "meta": {
    "title": ...,
    "author": ...,
    "description": ...,
    "generated_at": "YYYY-MM-DD HH:MM",
  },
  "comparison": {
    "baseline_key": ...,
    "split": ...,
    "mixed_splits": ...,
  },
  "models": [
    {
      "index": ...,
      "key": ...,
      "title_name": ...,
      "description": ...,
      "name": ...,
      "type": ...,
      "comparison_split": ...,
      "data_label": ...,
      "tuned_label": ...,
      "params": {...},
      "param_count": ...,
      "is_baseline": ...,
    },
  ],
  "metrics": [
    {
      "metric_id": ...,
      "metric_name": ...,
      "direction": "max" | "min",
      "keys": [...],
      "values": {...},
      "splits": {...},
      "deltas": {...},
    },
  ],
}
```

```
        "best_key": ...,
        "best_index": ...,
    },
],
}
```

Classification Example: Iris

Classification Setup

```
from pathlib import Path

from sklearn.datasets import load_iris
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from mlreport import ComparisonReport, Report

Path("reports").mkdir(exist_ok=True)
X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y,
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
author = "Lucas Summers"
theme = "light"
```

Appendix A: Classification Train/Test Split

```
split_model = GaussianNB()

split_model.fit(X_train, y_train)
y_pred_train = split_model.predict(X_train)
y_pred_test = split_model.predict(X_test)

split_report = Report(
    split_model,
    title="Appendix A: Iris Train/Test Split Report",
    author=author,
    description="Gaussian Naive Bayes evaluated on a standard train/test split.",
)
```

```

        theme=theme,
    )

    split_report.add_split("train", X_train, y_train, y_pred_train)
    split_report.add_split("test", X_test, y_test, y_pred_test)
    split_report.build().to_pdf("reports/iris_split_report.pdf").to_txt()

```

Appendix B: Classification Cross-Validation

```

cv_model = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", SVC(kernel="rbf", C=2.0, gamma="scale")),
    ]
)

cv_model.fit(X_train, y_train)

cv_report = Report(
    cv_model,
    title="Appendix B: Iris Cross-Validation Report",
    author=author,
    description="Support Vector Classifier evaluated with 5-fold stratified cross-validation.",
    theme=theme,
)

cv_report.add_crossval(X_train, y_train, cv=cv)
cv_report.build().to_pdf("reports/iris_cv_report.pdf").to_txt()

```

Appendix C: Classification Tuned Cross-Validation

```

search = GridSearchCV(
    estimator=GradientBoostingClassifier(random_state=42),
    param_grid={
        "n_estimators": [50, 100, 150],
        "learning_rate": [0.03, 0.1, 0.2],
    },
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,
)

search.fit(X_train, y_train)
best_tuned_model = search.best_estimator_

tuned_cv_report = Report(
    best_tuned_model,
    title="Appendix C: Iris Tuned Cross-Validation Report",
    author=author,
    description="Gradient Boosting tuned with GridSearchCV and evaluated with 5-fold CV.",
    theme=theme,
)

```

```
tuned_cv_report.add_crossval(X_train, y_train, cv=cv)
tuned_cv_report.add_search(search)
tuned_cv_report.build().to_pdf("reports/iris_tuned_cv_report.pdf").to_txt()
```

Appendix D: Classification Comparison

```
comparison = ComparisonReport(
    reports=[
        split_report,
        cv_report,
        tuned_cv_report,
    ],
    title="Appendix D: Iris Comparison Report",
    author=author,
    description="Comparison of three classification workflows on the Iris dataset.",
    theme=theme,
)

comparison.build().to_pdf("reports/iris_comparison_report.pdf", include_model_reports=False).to_txt()
```

Regression Example: Diabetes

Regression Setup

```
from pathlib import Path

from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV, KFold, train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from mlreport import ComparisonReport, Report

Path("reports").mkdir(exist_ok=True)

X, y = load_diabetes(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
)

cv = KFold(n_splits=5, shuffle=True, random_state=42)
author = "Lucas Summers"
theme = "light"
```

Appendix E: Regression Train/Test Split

```
split_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LinearRegression()),
])

split_model.fit(X_train, y_train)
y_pred_train = split_model.predict(X_train)
y_pred_test = split_model.predict(X_test)

split_report = Report(
    split_model,
    title="Appendix E: Diabetes Train/Test Split Report",
    author=author,
    description="Linear Regression evaluated on a standard train/test split.",
    theme=theme,
)

split_report.add_split("train", X_train, y_train, y_pred_train)
split_report.add_split("test", X_test, y_test, y_pred_test)
split_report.build().to_pdf("reports/diabetes_split_report.pdf").to_txt()
```

Appendix F: Regression Cross-Validation

```
cv_model = RandomForestRegressor(
    n_estimators=300,
    max_depth=6,
    random_state=42,
)

cv_model.fit(X_train, y_train)

cv_report = Report(
    cv_model,
    title="Appendix F: Diabetes Cross-Validation Report",
    author=author,
    description="Random Forest evaluated with 5-fold cross-validation.",
    theme=theme,
)

cv_report.add_crossval(X_train, y_train, cv=cv)
cv_report.build().to_pdf("reports/diabetes_cv_report.pdf").to_txt()
```

Appendix G: Regression Tuned Cross-Validation

```
search = GridSearchCV(
    estimator=Pipeline([
        ("scaler", StandardScaler()),
```

```

        ("model", KNeighborsRegressor()),
    ],
),
param_grid={
    "model__n_neighbors": [3, 5, 7, 9],
    "model__weights": ["uniform", "distance"],
},
cv=cv,
scoring="neg_root_mean_squared_error",
n_jobs=-1,
)

search.fit(X_train, y_train)
best_tuned_model = search.best_estimator_

tuned_cv_report = Report(
    best_tuned_model,
    title="Appendix G: Diabetes Tuned Cross-Validation Report",
    author=author,
    description="KNN regressor tuned with GridSearchCV and evaluated with 5-fold CV.",
    theme=theme,
)

tuned_cv_report.add_crossval(X_train, y_train, cv=cv)
tuned_cv_report.add_search(search)
tuned_cv_report.build().to_pdf("reports/diabetes_tuned_cv_report.pdf").to_txt()

```

Appendix H: Regression Comparison

```

comparison = ComparisonReport(
    reports=[
        split_report,
        cv_report,
        tuned_cv_report,
    ],
    title="Appendix H: Diabetes Comparison Report",
    author=author,
    description="Comparison of three regression workflows on the Diabetes dataset.",
    theme=theme,
)

comparison.build().to_pdf("reports/diabetes_comparison_report.pdf", include_model_reports=False).to_txt()

```

Appendix A: Classification Train/Test Split

Iris Classification Report - Train/Test Split

Lucas Summers
Logistic Regression evaluated on a standard train/test split.

MODEL OVERVIEW

Name	LogisticRegression
Type	Classification
Sklearn	True (v1.8.0)
Parameter Count	22

DATASET OVERVIEW

Features	4
Total Observations	150
CV Folds	None
Train	112 (74.7%)
Test	38 (25.3%)

Class Distribution

Class	Train	Test	Overall %
0	38	12	33.3%
1	37	13	33.3%
2	37	13	33.3%

HYPERPARAMETERS

memory	None	transform_input	None
verbose	False	scaler	StandardScaler
model	LogisticRegression	scaler__copy	True
scaler__with_mean	True	scaler__with_std	True
model__C	1.0	model__class_weight	None
model__dual	False	model__fit_intercept	True
model__intercept_scaling	1	model__l1_ratio	0.0
model__max_iter	1000	model__n_jobs	None
model__penalty	deprecated	model__random_state	None
model__solver	lbfgs	model__tol	0.0001
model__verbose	0	model__warm_start	False

METRICS (TRAIN/TEST SPLIT)

Metric	Train	Test	Gap
Accuracy	0.9643	0.9211	+0.0432
F1 Score (Macro)	0.9640	0.9230	+0.0410
F1 Score (Weighted)	0.9643	0.9209	+0.0434
Precision (Macro)	0.9640	0.9246	+0.0394
Precision (Weighted)	0.9643	0.9226	+0.0417
Recall (Macro)	0.9640	0.9231	+0.0409
Recall (Weighted)	0.9643	0.9211	+0.0432

*Gap = Train - Test. Large positive values may indicate overfitting.

METRICS (PER-CLASS)

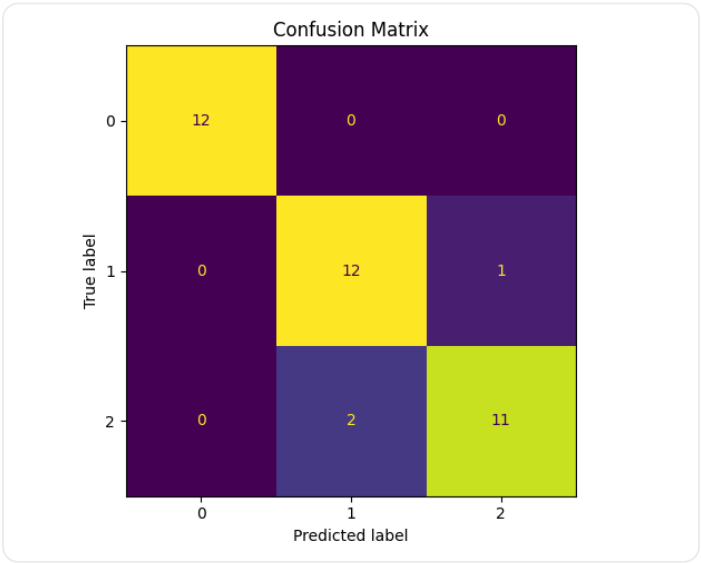
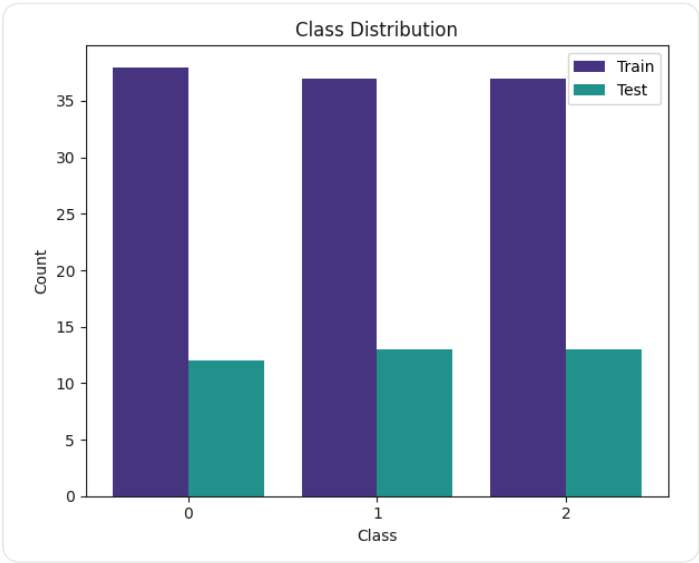
Train

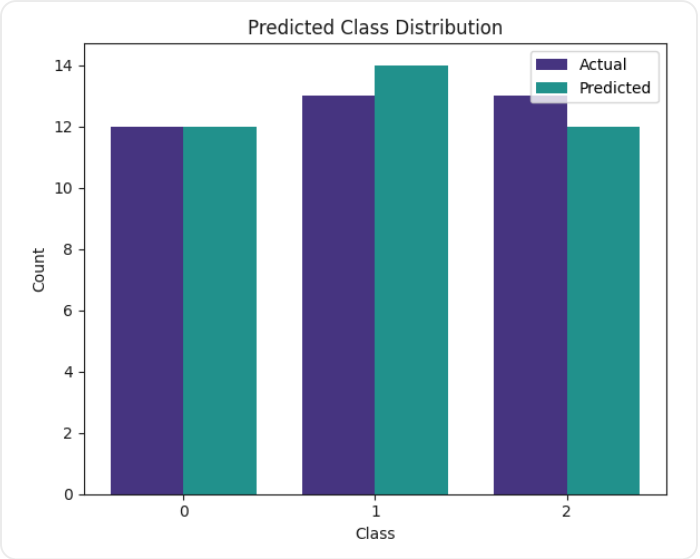
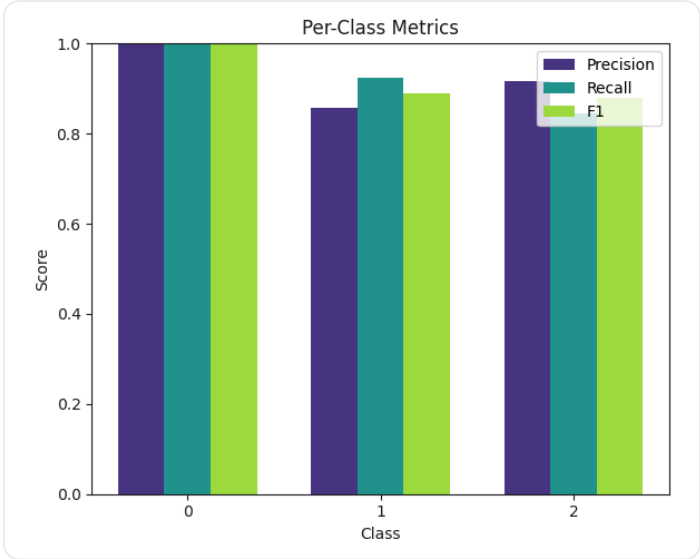
Class	Precision	Recall	F1
0	1.0000	1.0000	1.0000
1	0.9459	0.9459	0.9459
2	0.9459	0.9459	0.9459

Test

Class	Precision	Recall	F1
0	1.0000	1.0000	1.0000
1	0.8571	0.9231	0.8889
2	0.9167	0.8462	0.8800

VISUALIZATIONS





Appendix B: Classification Cross-Validation

Iris Classification Report - Cross-Validation

Lucas Summers
Random Forest evaluated with 5-fold stratified cross-validation.

MODEL OVERVIEW

Name	RandomForestClassifier
Type	Classification
Sklearn	True (v1.8.0)
Parameter Count	19

DATASET OVERVIEW

Features	4
Total Observations	112
CV Folds	5

Class Distribution

Class	CV	Overall %
0	38	33.9%
1	37	33.0%
2	37	33.0%

HYPERPARAMETERS

bootstrap	True	ccp_alpha	0.0
-----------	------	-----------	-----

class_weight	None	criterion	gini
max_depth	4	max_features	sqrt
max_leaf_nodes	None	max_samples	None
min_impurity_decrease	0.0	min_samples_leaf	1
min_samples_split	2	min_weight_fraction_leaf	0.0
monotonic_cst	None	n_estimators	200
n_jobs	None	oob_score	False
random_state	42	verbose	0
warm_start	False		

METRICS (CROSS-VALIDATION)

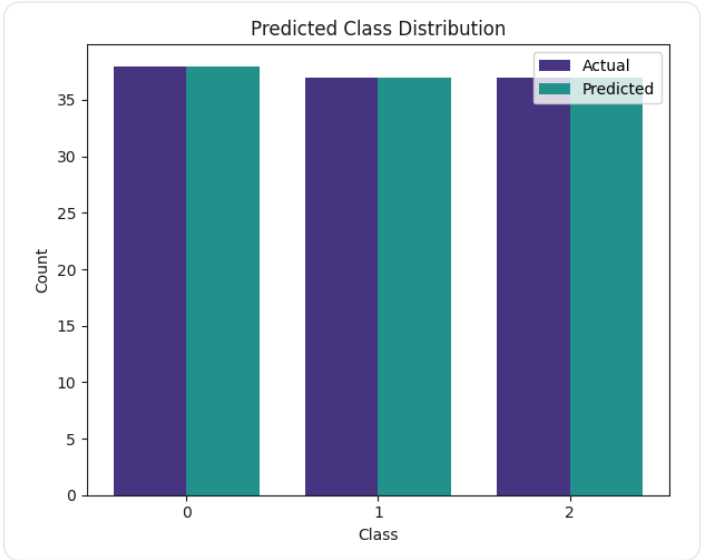
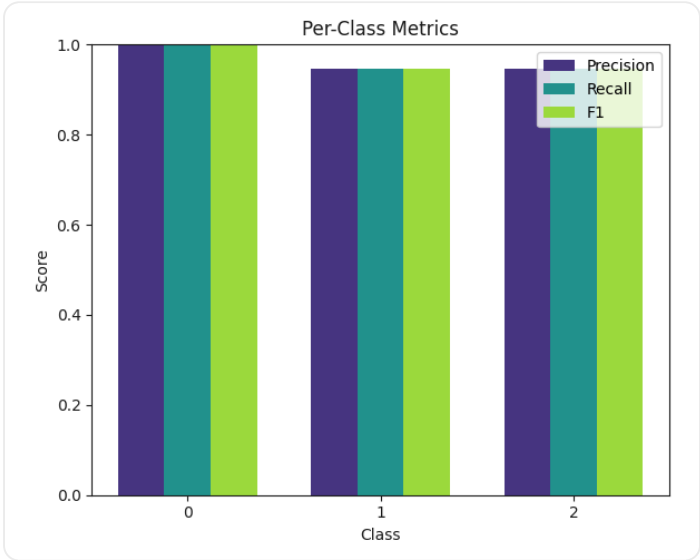
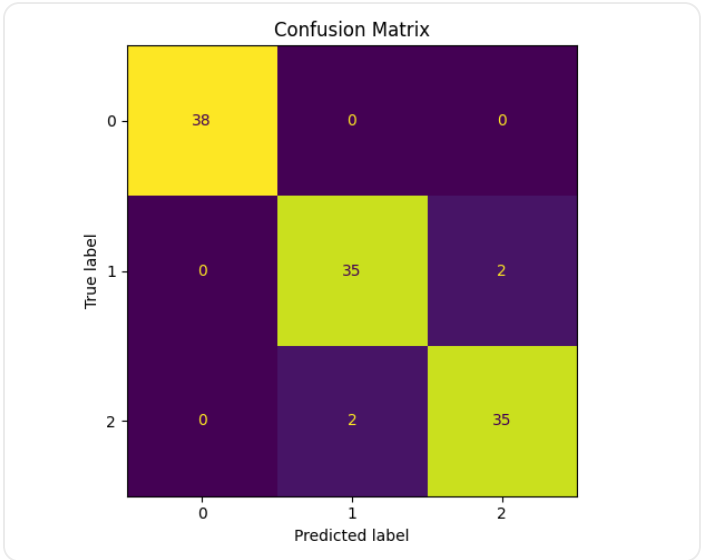
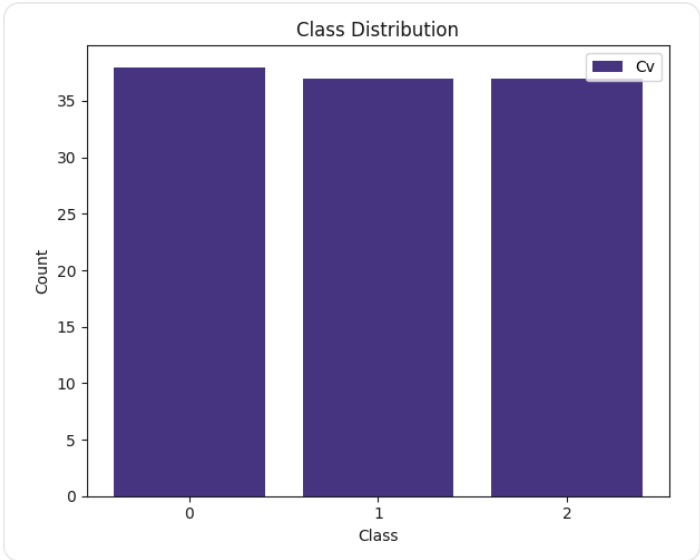
Metric	Mean	Std Dev	Min	Max
Accuracy	0.9644	0.0328	0.9130	1.0000
F1 Score (Macro)	0.9629	0.0347	0.9074	1.0000
F1 Score (Weighted)	0.9638	0.0336	0.9106	1.0000
Precision (Macro)	0.9709	0.0258	0.9333	1.0000
Precision (Weighted)	0.9701	0.0267	0.9304	1.0000
Recall (Macro)	0.9619	0.0356	0.9048	1.0000
Recall (Weighted)	0.9644	0.0328	0.9130	1.0000

METRICS (PER-CLASS)

Class	Precision	Recall	F1
-------	-----------	--------	----

0	1.0000	1.0000	1.0000
1	0.9459	0.9459	0.9459
2	0.9459	0.9459	0.9459

VISUALIZATIONS



Appendix C: Classification Tuned Cross-Validation

Iris Classification Report - Tuning + Cross-Validation

Lucas Summers
KNN tuned with GridSearchCV and evaluated with 5-fold CV.

MODEL OVERVIEW

Name	KNeighborsClassifier
Type	Classification
Sklearn	True (v1.8.0)
Parameter Count	16

DATASET OVERVIEW

Features	4
Total Observations	112
CV Folds	5

Class Distribution

Class	CV	Overall %
0	38	33.9%
1	37	33.0%
2	37	33.0%

HYPERPARAMETERS

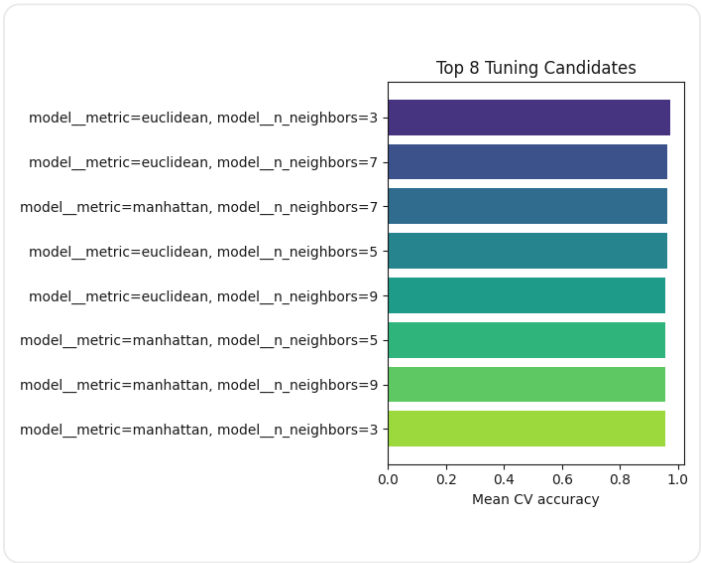
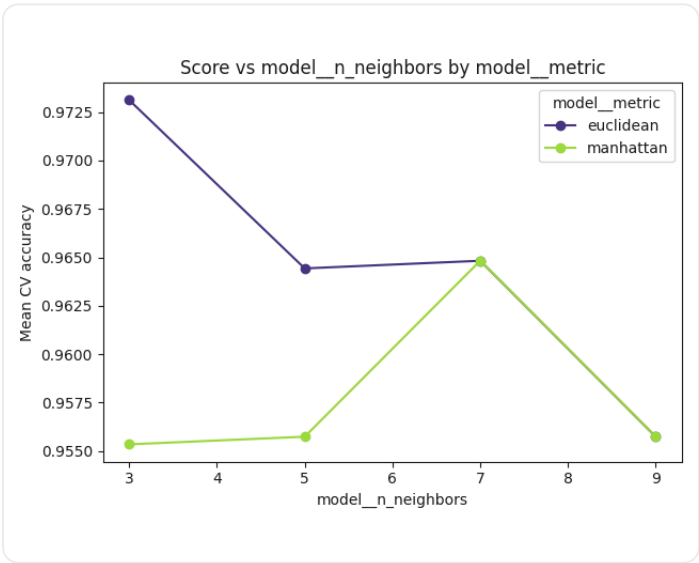
memory	None	transform_input	None
--------	------	-----------------	------

verbose	False	scaler	StandardScaler
model	KNeighborsClassifier	scaler__copy	True
scaler__with_mean	True	scaler__with_std	True
model__algorithm	auto	model__leaf_size	30
model__metric	euclidean	model__metric_params	None
model__n_jobs	None	model__n_neighbors	3
model__p	2	model__weights	uniform

Tuning Summary

Method	GridSearchCV
Metric	accuracy
CV Folds	5
Candidates	8
Best Score	0.9731

Tuning Plots



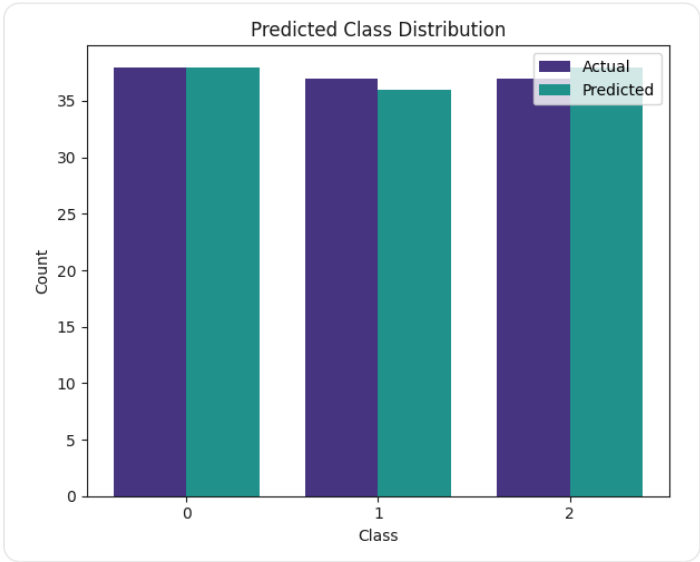
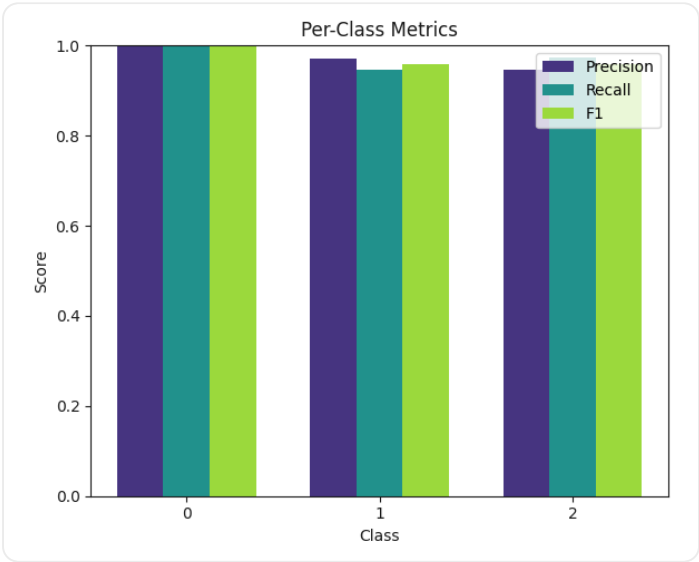
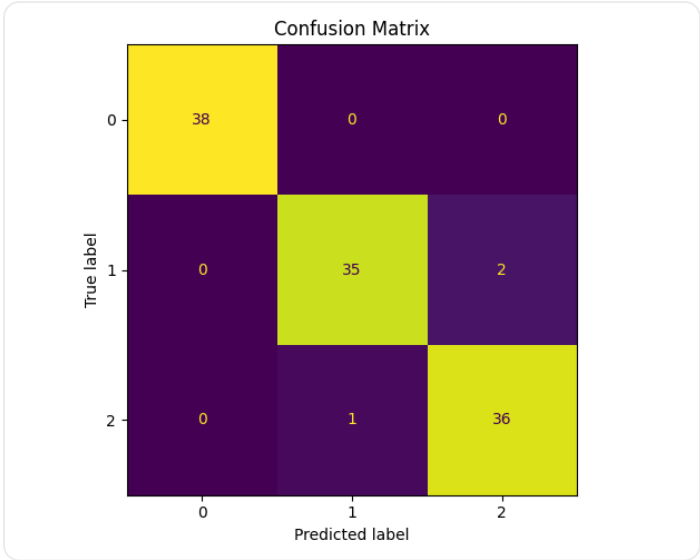
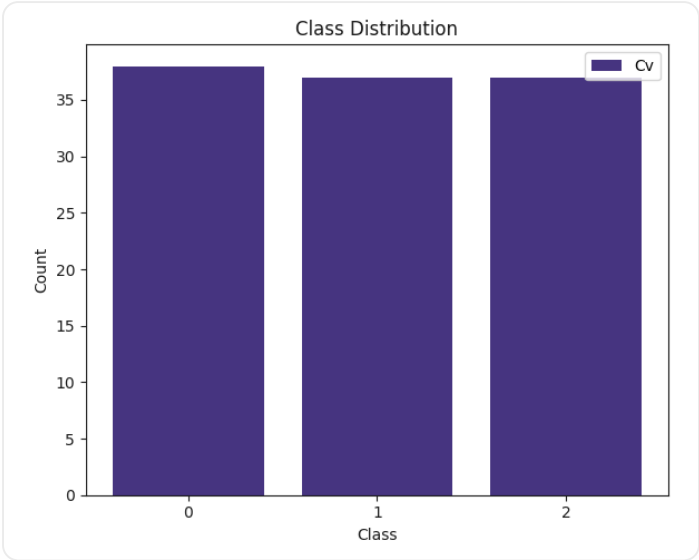
METRICS (CROSS-VALIDATION)

Metric	Mean	Std Dev	Min	Max
Accuracy	0.9731	0.0220	0.9545	1.0000
F1 Score (Macro)	0.9730	0.0220	0.9548	1.0000
F1 Score (Weighted)	0.9730	0.0221	0.9541	1.0000
Precision (Macro)	0.9769	0.0190	0.9583	1.0000
Precision (Weighted)	0.9762	0.0194	0.9596	1.0000
Recall (Macro)	0.9726	0.0225	0.9524	1.0000
Recall (Weighted)	0.9731	0.0220	0.9545	1.0000

METRICS (PER-CLASS)

Class	Precision	Recall	F1
0	1.0000	1.0000	1.0000
1	0.9722	0.9459	0.9589
2	0.9474	0.9730	0.9600

VISUALIZATIONS



Appendix D: Classification Comparison

Iris Workflow Comparison

Lucas Summers

Comparison of three classification workflows on the Iris dataset.

MODELS

MODEL 1 [BASELINE]

LogisticRegression

Description Logistic Regression evaluated on a standard train/test split.

Type Classification

Data Train/Test Split

Params 22

Tuned None

MODEL 2

RandomForestClassifier

Description Random Forest evaluated with 5-fold stratified cross-validation.

Type Classification

Data 5-fold CV

Params 19

Tuned None

MODEL 3

KNeighborsClassifier

Description KNN tuned with GridSearchCV and evaluated with 5-fold CV.

Type Classification

Data 5-fold CV

Params 16

Tuned 2 params

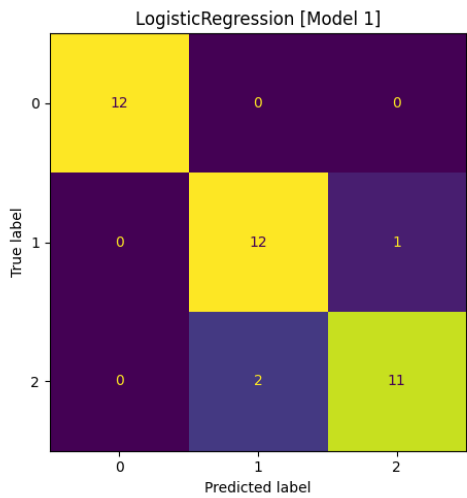
METRICS

Metric	Model 1	Model 2	Model 3	Best
Accuracy	0.9211	0.9644 (+0.0434)	0.9731 (+0.0521)	Model 3
F1 Score (Macro)	0.9230	0.9629 (+0.0399)	0.9730 (+0.0500)	Model 3
F1 Score (Weighted)	0.9209	0.9638 (+0.0429)	0.9730 (+0.0520)	Model 3
Precision (Macro)	0.9246	0.9709 (+0.0463)	0.9769 (+0.0522)	Model 3
Precision (Weighted)	0.9226	0.9701 (+0.0474)	0.9762 (+0.0536)	Model 3
Recall (Macro)	0.9231	0.9619 (+0.0388)	0.9726 (+0.0495)	Model 3
Recall (Weighted)	0.9211	0.9644 (+0.0434)	0.9731 (+0.0521)	Model 3

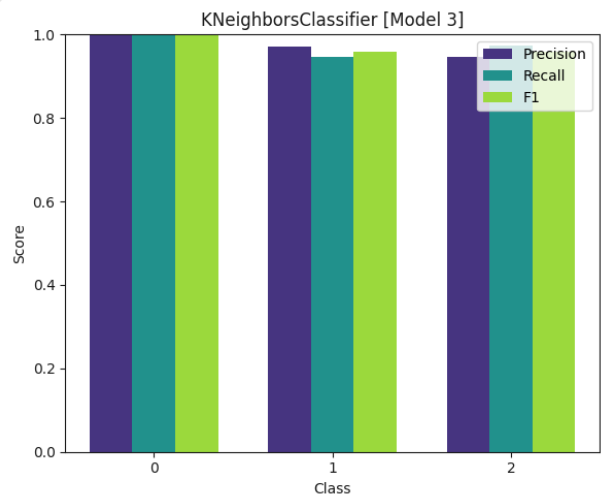
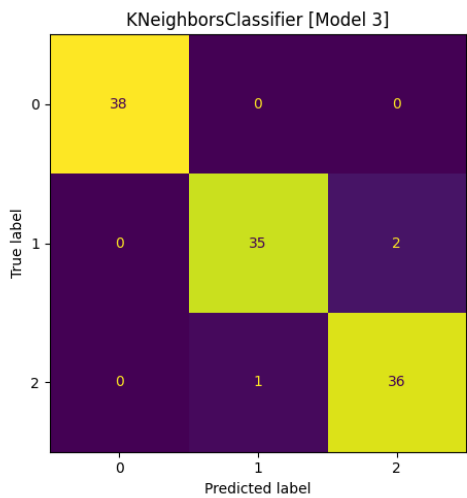
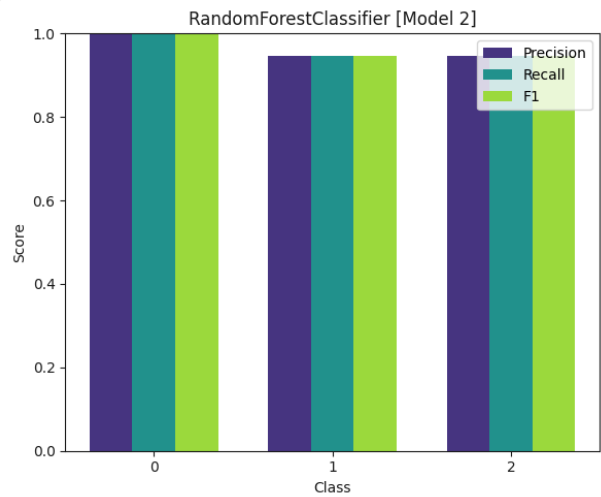
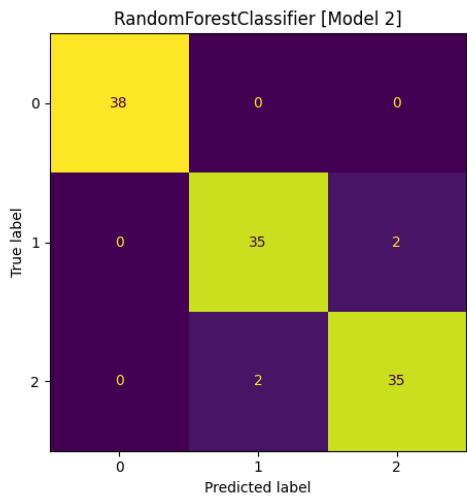
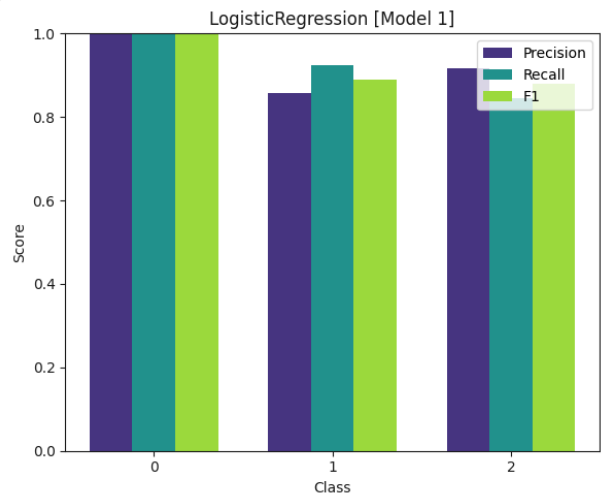
Metrics may be drawn from different evaluation splits across models.

VISUALIZATIONS

Confusion Matrix



Per-Class Metrics



Appendix E: Regression Train/Test Split

Diabetes Regression Report - Train/Test Split

Lucas Summers
Linear Regression evaluated on a standard train/test split.

MODEL OVERVIEW

Name	LinearRegression
Type	Regression
Sklearn	1.8.0
Parameter Count	13

DATASET OVERVIEW

Features	10
Total Observations	442
CV Folds	None
Train	331 (74.9%)
Test	111 (25.1%)

HYPERPARAMETERS

memory	None	transform_input	None
verbose	False	scaler	StandardScaler
model	LinearRegression	scaler__copy	True
scaler__with_mean	True	scaler__with_std	True

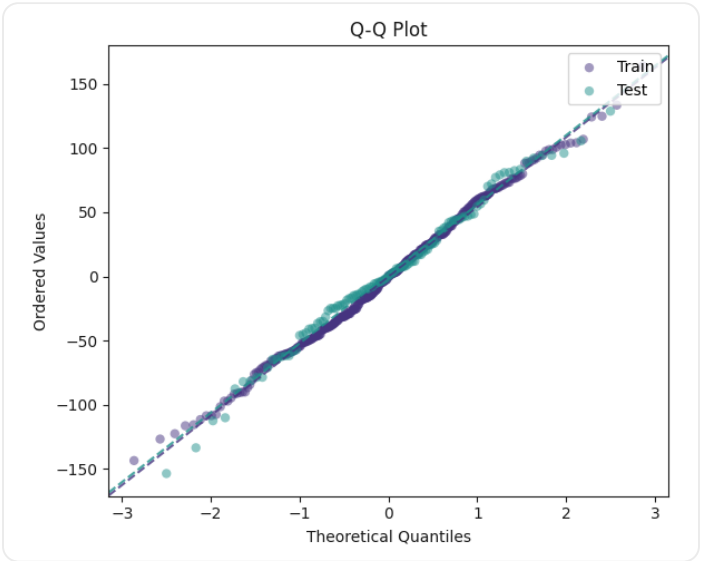
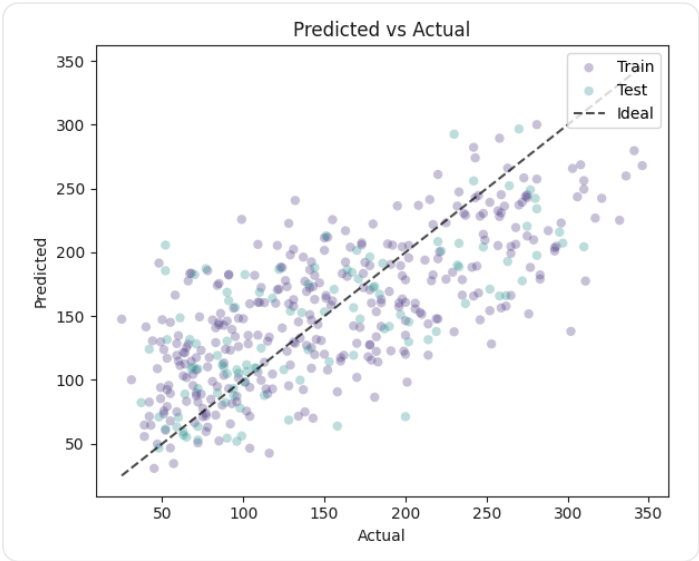
model__copy_X	True	model__fit_intercept	True
model__n_jobs	None	model__positive	False
model__tol	1e-06		

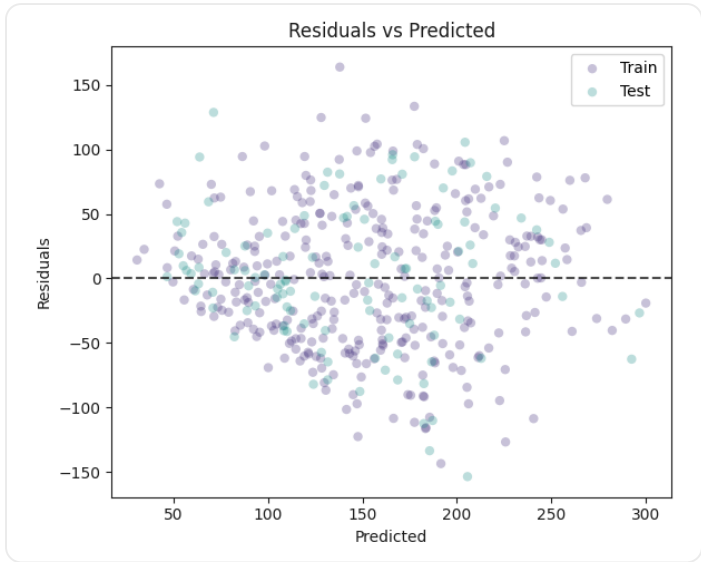
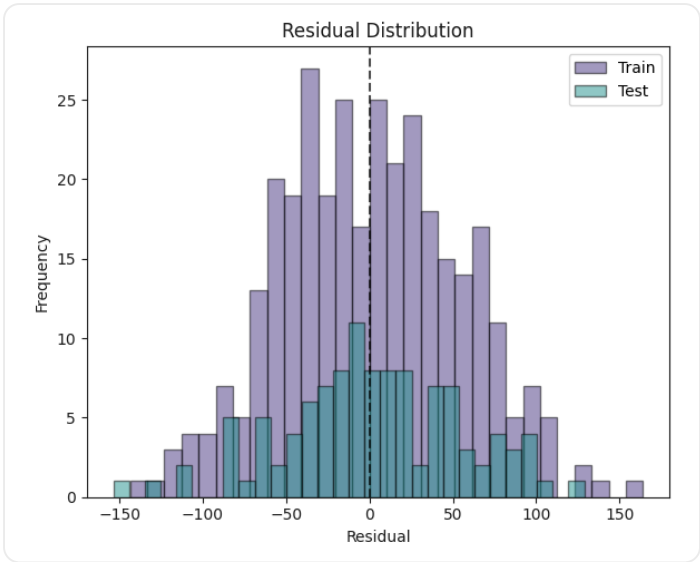
METRICS (TRAIN/TEST SPLIT)

Metric	Train	Test	Gap
Mean Absolute Error	44.0548	41.5485	+2.5063
Max Error	163.9605	153.6572	+10.3033
Median Absolute Error	38.9980	35.2108	+3.7872
Mean Squared Error	2907.2578	2848.3107	+58.9471
R ² Score	0.5190	0.4849	+0.0341
Root Mean Squared Error	53.9190	53.3696	+0.5494

*Gap = Train - Test. Large positive values may indicate overfitting.

VISUALIZATIONS





Appendix F: Regression Cross-Validation

Diabetes Regression Report - Cross-Validation

Lucas Summers
Random Forest evaluated with 5-fold cross-validation.

MODEL OVERVIEW

Name	RandomForestRegressor
Type	Regression
Sklearn	1.8.0
Parameter Count	18

DATASET OVERVIEW

Features	10
Total Observations	331
CV Folds	5

HYPERPARAMETERS

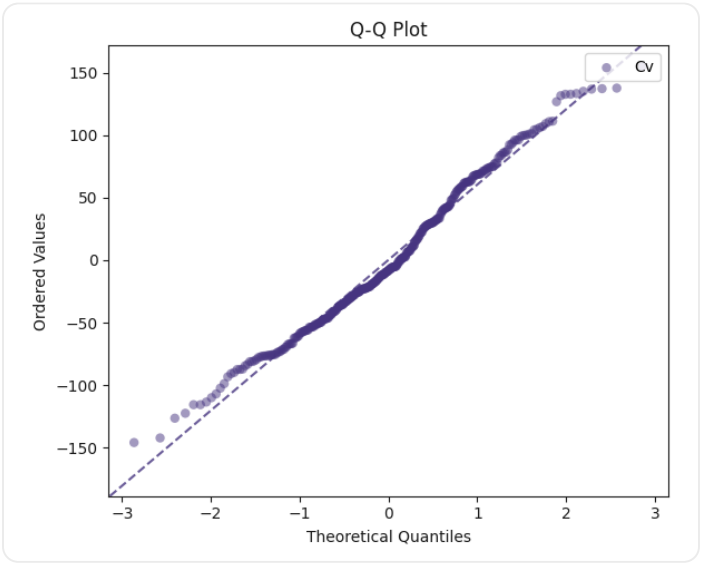
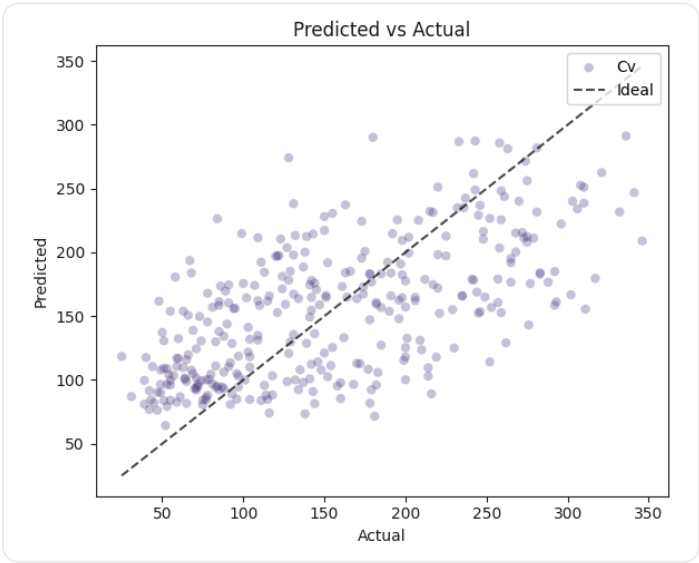
bootstrap	True	ccp_alpha	0.0
criterion	squared_error	max_depth	6
max_features	1.0	max_leaf_nodes	None
max_samples	None	min_impurity_decrease	0.0
min_samples_leaf	1	min_samples_split	2
min_weight_fraction_leaf	0.0	monotonic_cst	None

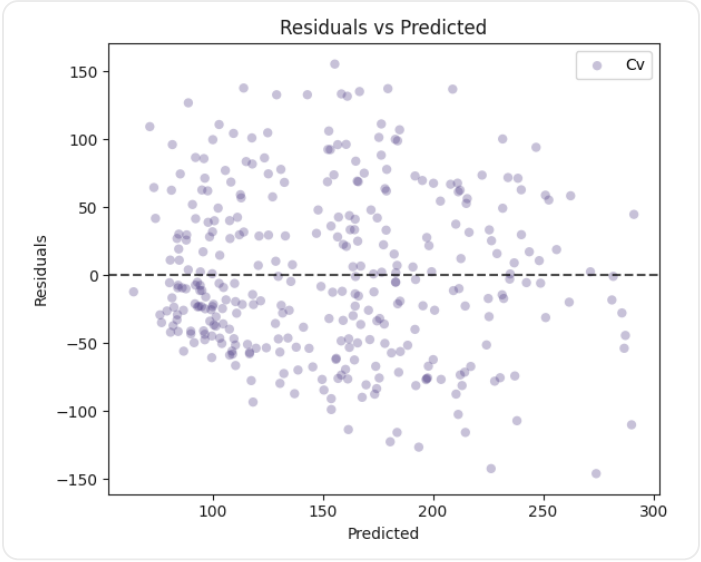
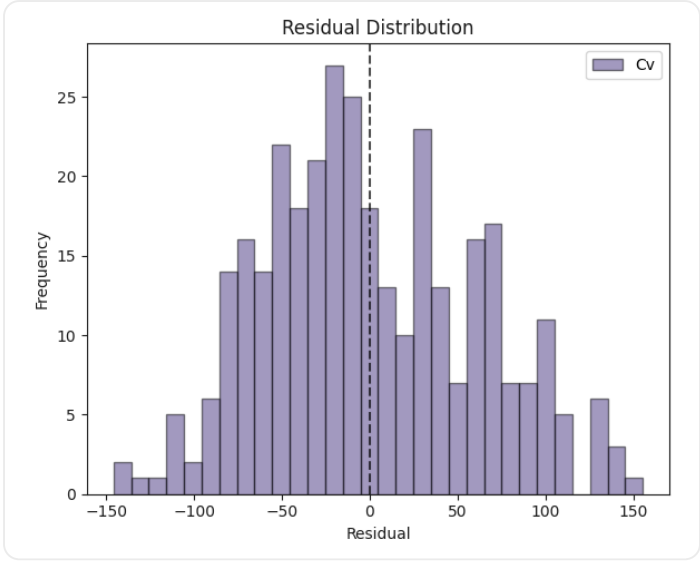
n_estimators	300	n_jobs	None
oob_score	False	random_state	42
verbose	0	warm_start	False

METRICS (CROSS-VALIDATION)

Metric	Mean	Std Dev	Min	Max
Mean Absolute Error	49.7012	3.0789	45.0964	54.1114
Max Error	142.7364	7.9678	131.8291	155.5259
Median Absolute Error	44.9407	5.2820	39.9127	54.8522
Mean Squared Error	3637.4399	414.3230	3033.6358	4183.8147
R ² Score	0.3763	0.1192	0.1523	0.4779
Root Mean Squared Error	60.2124	3.4502	55.0785	64.6824

VISUALIZATIONS





Appendix G: Regression Tuned Cross-Validation

Diabetes Regression Report - Tuning + Cross-Validation

Lucas Summers
KNN regressor tuned with GridSearchCV and evaluated with 5-fold CV.

MODEL OVERVIEW

Name	KNeighborsRegressor
Type	Regression
Sklearn	1.8.0
Parameter Count	16

DATASET OVERVIEW

Features	10
Total Observations	331
CV Folds	5

HYPERPARAMETERS

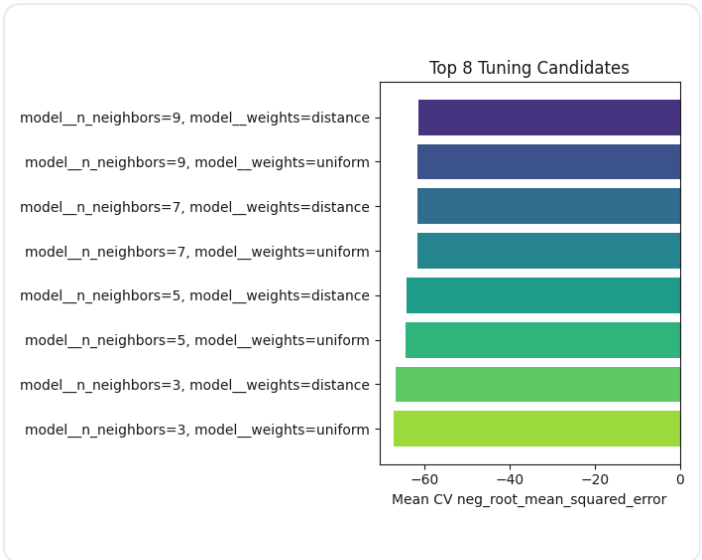
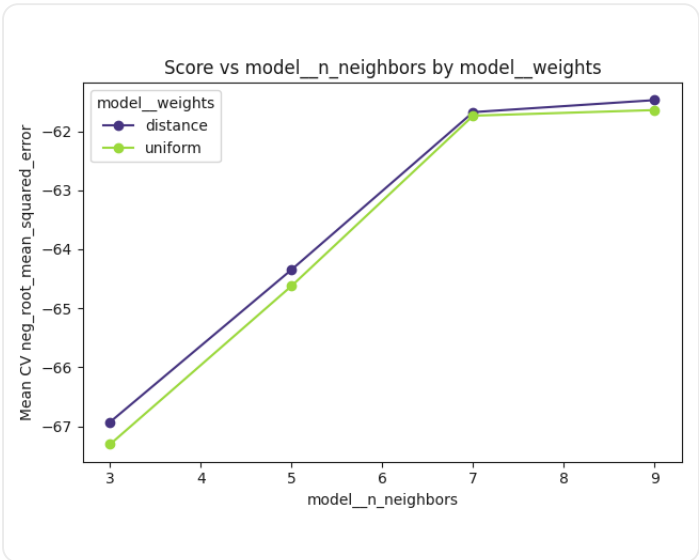
memory	None	transform_input	None
verbose	False	scaler	StandardScaler
model	KNeighborsRegressor	scaler__copy	True
scaler__with_mean	True	scaler__with_std	True
model__algorithm	auto	model__leaf_size	30
model__metric	minkowski	model__metric_params	None

model__n_jobs	None	model__n_neighbors	9
model__p	2	model__weights	distance

Tuning Summary

Method	GridSearchCV
Metric	neg_root_mean_squared_error
CV Folds	5
Candidates	8
Best Score	-61.4718

Tuning Plots

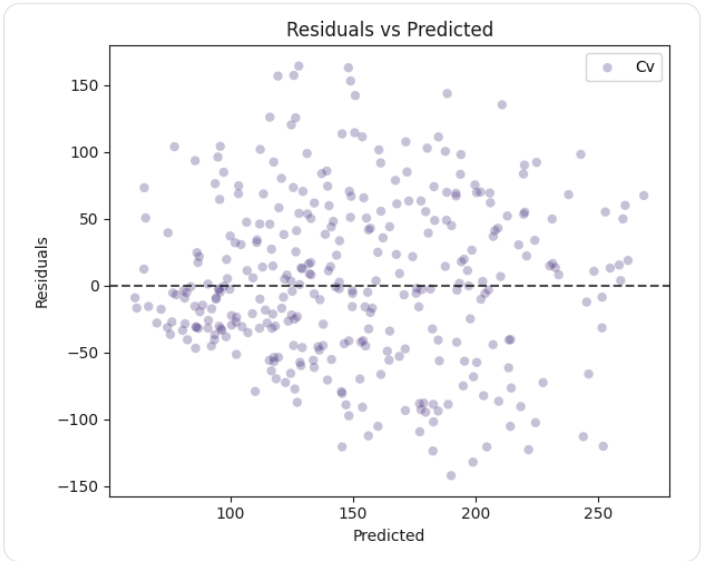
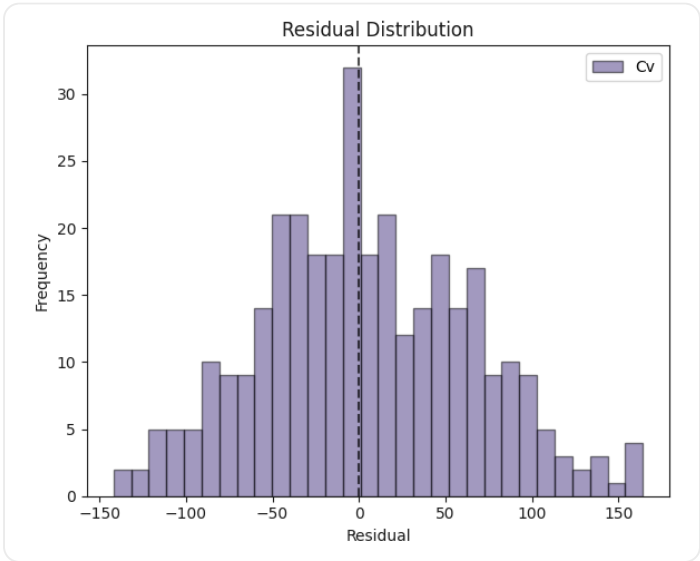
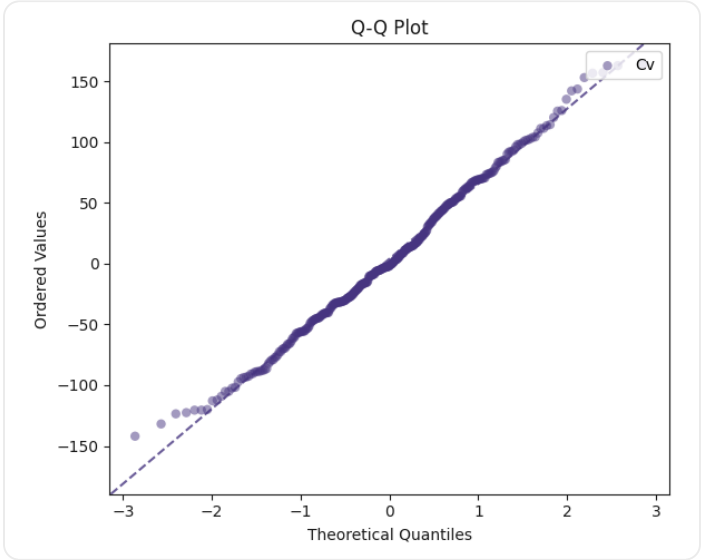
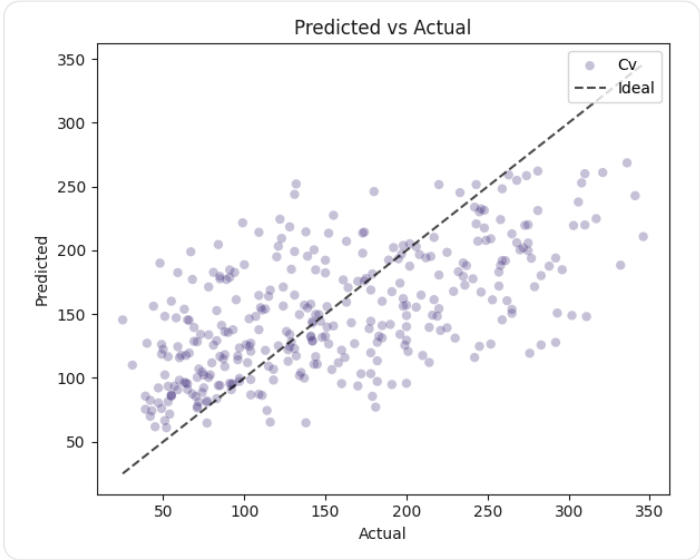


METRICS (CROSS-VALIDATION)

Metric	Mean	Std Dev	Min	Max
Mean Absolute Error	49.3585	3.0074	44.3420	52.6329
Max Error	155.8950	7.9642	142.1326	164.1776
Median Absolute Error	42.9693	5.3624	33.8980	49.0457
Mean Squared Error	3790.9658	417.7825	3045.2140	4163.9169

R^2 Score	0.3535	0.0975	0.1815	0.4759
Root Mean Squared Error	61.4718	3.4901	55.1835	64.5284

VISUALIZATIONS



Appendix H: Regression Comparison

Diabetes Workflow Comparison

Lucas Summers

Comparison of three regression workflows on the Diabetes dataset.

MODELS

MODEL 1 [BASELINE]

LinearRegression

Description Linear Regression evaluated on a standard train/test split.

Type Regression

Data Train/Test Split

Params 13

Tuned None

MODEL 2

RandomForestRegressor

Description Random Forest evaluated with 5-fold cross-validation.

Type Regression

Data 5-fold CV

Params 18

Tuned None

MODEL 3

KNeighborsRegressor

Description KNN regressor tuned with GridSearchCV and evaluated with 5-fold CV.

Type Regression

Data 5-fold CV

Params 16

Tuned 2 params

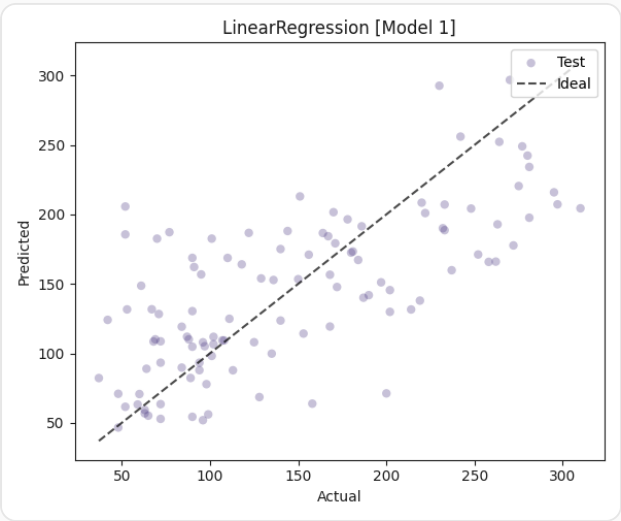
METRICS

Metric	Model 1	Model 2	Model 3	Best
Mean Absolute Error	41.5485	49.7012 (+8.1527)	49.3585 (+7.8100)	Model 2
Max Error	153.6572	142.7364 (-10.9208)	155.8950 (+2.2378)	Model 3
Median Absolute Error	35.2108	44.9407 (+9.7299)	42.9693 (+7.7585)	Model 2
Mean Squared Error	2848.3107	3637.4399 (+789.1292)	3790.9658 (+942.6551)	Model 3
R ² Score	0.4849	0.3763 (-0.1086)	0.3535 (-0.1314)	Model 1
Root Mean Squared Error	53.3696	60.2124 (+6.8429)	61.4718 (+8.1023)	Model 3

Metrics may be drawn from different evaluation splits across models.

VISUALIZATIONS

Predicted vs Actual



Residual Distribution

